



# Langage SQL

## Séquence 3 : Langage de Manipulation de Données (DML)

### Table des matières

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>La commande INSERT</b>                          | <b>2</b> |
| <b>2</b> | <b>La commande DELETE et UPDATE</b>                | <b>3</b> |
| 2.1      | La clause WHERE .....                              | 3        |
| 2.2      | La commande DELETE .....                           | 4        |
| 2.3      | La commande UPDATE .....                           | 4        |
| <b>3</b> | <b>Extraction d'informations : commande SELECT</b> | <b>6</b> |
| 3.1      | Les clauses ORDER BY et GROUP BY .....             | 6        |
| 3.2      | La clause HAVING .....                             | 7        |
| 3.3      | Notion d'alias .....                               | 8        |
| 3.4      | Opérateurs de Jointures .....                      | 9        |
| 3.5      | Opérateurs IN et NOT IN .....                      | 10       |
| 3.6      | Opérateurs IS NULL et IS NOT NULL .....            | 10       |
| 3.7      | Opérateurs ensemblistes .....                      | 10       |
| 3.8      | Les fonctions de groupe ou d'agrégation .....      | 11       |

---

## Introduction

**DML** (Data Management Language ou simplement (Langage de Manipulation de données) permet de gérer, manipuler les éléments d'une base de données. **SQL** définit les opérations suivantes :

+ Opérations reposant sur la théorie des ensembles :

1. Union ( n-aire )
2. Intersection ( n-aire )
3. Différence ( binaire )
4. Produit cartésien ( binaire )

+ Opérations spécifiques aux Bases de données relationnelles :

1. Sélection ( un-aire )
2. Projection ( un-aire )
3. Jointure ( binaire )

## 1 La commande INSERT

Dans l'alimentation d'une base de données relationnelle, les données sont saisies dans une table par la commande **INSERT INTO**. Elle permet d'ajouter des tuples à une relation. Les attributs doivent être listés dans le même ordre que dans la commande **CREATE TABLE**. Dans la figure 1 , on a inséré un auteur dans la table musique. Dans cette requête, comme on insert dans toutes les colonnes, les noms des attributs ne pas obligatoires. Les types aussi doivent être respectés.

Dans la figure 2, on note qu'il est possible d'ajouter plusieurs ligne avec la même commande **INSERT**.

S'il y a des attributs supposés avoir des valeurs non connues. Elles prennent la valeur **NULL** par défaut. **INSERT** doit respecter les contraintes d'intégrité référentielle. Par exemple, la table *Selection* n'a de sens que si on a une *musique* et un *film*. Alors insérer une musique où film inexistant va générer des erreurs.

**SQL** vérifie en contrepartie qu'une valeur nulle ne peut pas être spécifiée pour un attribut pour lequel elle n'a pas été autorisée à la création de la table (commande **CREATE TABLE** et l'option **NOT NULL**).

```

+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| IdFilm     | decimal(10,0) | NO   |     | NULL    |       |
| annee      | date          | YES  |     | NULL    |       |
| Titre      | varchar(255)  | YES  |     | NULL    |       |
| categorie  | varchar(100)  | YES  |     | NULL    |       |
| duree      | decimal(10,0) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
5 rows in set (0.02 sec)

mysql> ALTER TABLE Film ADD CONSTRAINT pk_Film PRIMARY KEY (IdFilm);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> describe Film;
+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| IdFilm     | decimal(10,0) | NO   | PRI | NULL    |       |
| annee      | date          | YES  |     | NULL    |       |
| Titre      | varchar(255)  | YES  |     | NULL    |       |
| categorie  | varchar(100)  | YES  |     | NULL    |       |
| duree      | decimal(10,0) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

```

Figure 1 – Ajout d'une ligne dans la table Musique

```

INSERT INTO Musique (IdMusique,nomAuteur,prenomAuteur,emailauteur)
values (145,"NDIAYE","Samba","Samba@gmail.com"),(146,"NDIAYE","Jeanne","jeanne@gmail.com"),
(149,"Diop","samour","Samour@gmail.com"),(130,"Gaye","Mbaye","gaye@gmail.com"),
(131,"diop","Maimouna","Samba@gmail.com"),(132,"NDIAYE","Deme","deme@gmail.com"),
(133,"lo","mami","Samba@gmail.com"),(134,"diop","dieynaba","Samba@gmail.com");

```

Figure 2 – Ajout de plusieurs lignes

## 2 La commande DELETE et UPDATE

Dans les bases de données, dès fois on a besoin de supprimer des lignes ou de les modifier. SQL nous donne la possibilité de mettre en œuvre ces deux opérations. Ces dernières vont être appliquées sur toutes les lignes si on ne met pas un filtre (un prédicat) **WHERE**.

### 2.1 La clause WHERE

Cette clause est toujours suivie d'une condition de recherche pour chaque ligne (enregistrement) résultant de la clause FROM. La table résultante contient les lignes (enregistrements) répondant VRAI à la condition de re-

---

## 2.2 La commande DELETE

cherche. Elle réalise l'opération de *sélection* et spécifie les conditions particulières (un filtre). La condition appelée prédicat prend la valeur VRAI ou FAUX. Les opérateurs utilisables dans le prédicat peuvent être de type :

- + arithmétique ( =, !=, <, >, <=, >=)
- + intervalle ( BETWEEN),
- + texte ( LIKE),
- + ensemble de valeurs (IN, NOT IN),
- + test de nullité (IS NULL | IS NOT NULL)

## 2.2 La commande DELETE

Les lignes sont supprimées d'une table par la commande **DELETE**. Il ne faut pas confondre cette commande avec DROP qui fait aussi une suppression mais des les DDL (suppression de tables, de contraintes, etc.) Structure de la commande :

**DELETE FROM** <nom table> **WHERE** condition

Plusieurs lignes peuvent être supprimées par la commande **DELETE**. Vous pouvez même, si vous n'y prenez garde, effacer toutes les lignes d'une table : il suffit que :

- + votre condition soit toujours vraie quel que soit la ligne de la table, + qu'il n'y ait pas de conditions.

## 2.3 La commande UPDATE

La commande **UPDATE** : permet de modifier certaines valeurs d'attributs dans un ou plusieurs tuples.

Structure de la commande

**UPDATE** <nom table> **SET** <colonne1> = <expression>, < colonne 2> = <expression2> **WHERE** condition

Plusieurs lignes peuvent être modifiées par une seule commande **UPDATE**. Exécuter cette comment revient à indiquer :

---

+ le nom de la table ou de la vue (qui n'est pas l'objet du cours) + le nom des attribues à modifier avec leur valeur :

+ soit la nouvelle valeur,

+ soit une expression permettant de déterminer la nouvelle valeur affectée à la zone.

+ la ou les conditions de prise en compte : derrière la close **WHERE** qui n'est pas obligatoire

### 2.3 La commande UPDATE

Exemples d'utilisations : Dans la figure 3, nous avons toutes les lignes de la table *Musique*. On constate que le email de Dieynaba Diop n'est pas correcte.

On lui a donné le Email de Samba Ndiaye. Dans la figure 4, on a d'abord

```
mysql> select * from Musique;
+-----+-----+-----+-----+
| IdMusique | nomAuteur | prenomAuteur | emailAuteur |
+-----+-----+-----+-----+
| 144 | SALL | Moussa | moussa@gmail.com |
| 145 | NDIAYE | Samba | Samba@gmail.com |
| 146 | NDIAYE | Jeanne | jeanne@gmail.com |
| 149 | Diop | samour | Samour@gmail.com |
| 130 | Gaye | Mbaye | gaye@gmail.com |
| 131 | diop | Maimouna | Samba@gmail.com |
| 132 | NDIAYE | Deme | deme@gmail.com |
| 133 | lo | mami | Samba@gmail.com |
| 134 | diop | dieynaba | Samba@gmail.com |
+-----+-----+-----+-----+
9 rows in set (0.02 sec)
```

Figure 3 – État initial des lignes de la table Musique

la commande UPDATE. Pour fixer la ligne de Dieynaba, on doit utiliser la clause **WHERE** avec comme condition *EmailAuteur=134* qui correspond avec l'IdMusique de Dieynaba Diop. Dans cette même figure, on la vue de la table après modification.

```
mysql> UPDATE Musique set emailauteur="dieynaba@gmail.com" WHERE IdMusique=134;
Query OK, 1 row affected (0.08 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from Musique;
+-----+-----+-----+-----+
| IdMusique | nomAuteur | prenomAuteur | emailAuteur |
+-----+-----+-----+-----+
| 144 | SALL | Moussa | moussa@gmail.com |
| 145 | NDIAYE | Samba | Samba@gmail.com |
| 146 | NDIAYE | Jeanne | jeanne@gmail.com |
| 149 | Diop | samour | Samour@gmail.com |
| 130 | Gaye | Mbaye | gaye@gmail.com |
| 131 | diop | Maimouna | Samba@gmail.com |
| 132 | NDIAYE | Deme | deme@gmail.com |
| 133 | lo | mami | Samba@gmail.com |
| 134 | diop | dieynaba | dieynaba@gmail.com |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

Figure 4 – La commande UPDATE et l'état de la table après modification

### 3 Extraction d'informations : commande SELECT

**SELECT** est une commande SQL qui permet d'extraire des données d'une base de données relationnelle. Une commande **SELECT** peut obtenir zéro ou plusieurs tuples (lignes) provenant des tables et des vues. Dû à la nature déclarative du langage SQL, une commande **SELECT** décrit un jeu de résultat voulus, et non la manière de les obtenir. Cette commande est basée sur l'algèbre relationnelle, en effectuant des opérations de sélection de données sur plusieurs tables relationnelles par projection.

Sa syntaxe est la suivante :

**SELECT** [**ALL**] | [**DISTINCT**] \* **FROM** <Liste des tables> [**WHERE** <condition logique>];

L'option **ALL** est, par opposition à l'option **DISTINCT**, l'option par défaut. Elle permet de sélectionner l'ensemble des lignes satisfaisant à la condition logique. L'option **DISTINCT** permet de ne conserver que des lignes distinctes, en éliminant les doublons. La liste des noms de colonnes indique la liste des colonnes choisies, séparées par des virgules. Lorsque l'on désire sélectionner l'ensemble des colonnes d'une table il n'est pas nécessaire de saisir la liste de ses colonnes, l'option \* permet de réaliser cette tâche. La liste des tables indique l'ensemble des tables (séparées par des virgules) sur lesquelles on opère. La condition *logique* permet d'exprimer des qualifications complexes à l'aide d'opérateurs logiques et de comparateurs arithmétiques. Il existe d'autres options pour la commande **SELECT** :

+ **GROUP BY** +  
**ORDER BY**

---

### + HAVING

Dans les figures 5 et 6, on trouve l'utilisation de **ALL** et de **DISTINCT**. Dans la première figure, tous les tuples de la colonne NomAuteur sont sélectionnés. Dans la deuxième figure, on a utilisé l'option **DISTINCT**, alors toutes les répétitions sont supprimées.

## 3.1 Les clauses **ORDER BY** et **GROUP BY**

Cette clause est facultative avec l'ordre **SELECT**. Si elle est indiquée, elle doit apparaître après les clauses **FROM** ET **WHERE**. Elle classe les lignes (enregistrements) de la table (fichier) résultante selon les noms des colonnes (zones) que vous avez identifiées. Si vous avez identifié plus d'une colonne (zone), les lignes (enregistrements) sont d'abord classées selon le

### 3.2 La clause *HAVING*

```
mysql> SELECT (NomAuteur) FROM Musique;
+-----+
| NomAuteur |
+-----+
| SALL      |
| NDIAYE    |
| NDIAYE    |
| Diop      |
| Gaye      |
| diop      |
| NDIAYE    |
| lo        |
| diop      |
+-----+
9 rows in set (0.03 sec)
```

Figure 5 – Sélection plus projection avec l'option **ALL**

```
mysql> SELECT DISTINCT (NomAuteur) FROM Musique;
+-----+
| NomAuteur |
+-----+
| SALL      |
| NDIAYE    |
| Diop      |
| Gaye      |
| lo        |
+-----+
5 rows in set (0.03 sec)
```

Figure 6 – Sélection plus projection avec l'option **DISTINCT**

nom des colonnes (zones) que vous avez identifié en premier, puis selon le nom des colonnes (zones) que vous avez identifié en second, etc. Elle permet de spécifier l'ordre d'affichage des lignes. La clause **ORDER BY** permet l'établissement d'un ordre des tuples.

+ ASC : ordre ascendant

+ DEC : ordre descendant

Dans la figure 7, on a un exemple de requête avec ORDER BY. Ici, on voit que les tuples sont triés par ordre croissant de leur nom.

La clause **GROUP BY** permet le regroupement de tuples (lignes). Tous les attributs utilisés dans la clause **SELECT** qui ne sont pas des agrégations doivent obligatoirement se retrouver dans la clause **GROUP BY**.

## 3.2 La clause HAVING

La clause **HAVING** permet de sélectionner un sous ensemble de tous les groupes obtenus par la clause **GROUP BY**. Elle ne peut être utilisée que si

### 3.3 Notion d'alias

```
mysql> SELECT * FROM Musique ORDER BY NomAuteur;
+-----+-----+-----+-----+
| IdMusique | nomAuteur | prenomAuteur | emailAuteur |
+-----+-----+-----+-----+
| 149 | Diop | samour | Samour@gmail.com |
| 131 | diop | Maimouna | Samba@gmail.com |
| 134 | diop | dieynaba | dieynaba@gmail.com |
| 130 | Gaye | Mbaye | gaye@gmail.com |
| 133 | lo | mami | Samba@gmail.com |
| 145 | NDIAYE | Samba | Samba@gmail.com |
| 146 | NDIAYE | Jeanne | jeanne@gmail.com |
| 132 | NDIAYE | Deme | deme@gmail.com |
| 144 | SALL | Moussa | moussa@gmail.com |
+-----+-----+-----+-----+
9 rows in set (0.03 sec)
```

Figure 7 – Exemple de select avec ORDER BY

la clause **GROUP BY** est utilisée.

## 3.3 Notion d'alias

Les alias sont des noms de remplacement que l'on donne de manière temporaire (le temps d'une requête) à une colonne, une table, une donnée. Les



alias sont introduits par le mot-clé **AS**. Ce mot-clé est facultatif, vous pouvez très bien définir un alias sans utiliser **AS**. Par exemple, si vous lancez la requête *SELECT 10+5* ; vous aurez une table temporaire dont le nom de la colonne est "10+5". Par contre, si vous lancez la *SELECT 10+5 AS somme* ; , vous aurez une table temporaire dont le nom de la colonne est *somme* qui va représenter l'alias de la colonne. Dans la figure 8, vous avez un exemple d'utilisation de cet alias.

```
mysql> select 3+8;
+-----+
| 3+8 |
+-----+
| 11 |
+-----+
1 row in set (0.00 sec)

mysql> select 3+8 AS somme;
+-----+
| somme |
+-----+
| 11 |
+-----+
1 row in set (0.00 sec)
```

Figure 8 – Utilisation des alias

### 3.4 Opérateurs de Jointures

## 3.4 Opérateurs de Jointures

Un des principaux intérêts d'une base de données est de pouvoir créer des relations entre les tables, de pouvoir les lier entre elles. Le principe des jointures est plutôt simple à comprendre et indispensable. Elles vont vous permettre de jongler avec plusieurs tables dans la même requête. Il existe plusieurs types de jointures telles que la jointure naturelle, la Jointure interne

Jointure externe, d'équi-jointure, etc. dans ce cours, nous verrons la jointure.

### Jointure naturelle

La jointure naturelle consiste à opérer une jointure avec une condition d'égalité. Cette condition d'égalité dans la jointure qui se porte nécessairement sur les clefs (primaires et étrangères). Si la condition ne se porte pas sur les clés, on parle d'équi-jointure.

Dans le schéma **GestionFilm**, on veut stocker le pays d'origine des auteurs, acteurs et producteurs. Dans ces trois relations, on va ajouter la clé de la relation Pays qui sera une clé étrangère dans chacune de ces trois.

Pays(**CodePays**,libelle)

Musique(**IdMusique**,NomAuteur,PrenomAuteur,EmailAuteur,#CodePays)

Producteur(**IdProducteur**,NomProducteur,PrenomProducteur,EmailProducteur,#CodePays)

Acteur(**IdActeur**,NomActeur,PrenomActeur,EmailActeur,#CodePays)

*Par exemple, nous voulons par exemple afficher tous auteurs avec leur pays d'origine.*

Ici, nous constatons que le libellé du pays et les informations de l'auteur sont dans deux tables différentes.

```
mysql> Select M.nomAuteur,M.prenomAuteur,P.CodePays,P.libelle
-> from Musique M, Pays P
-> where P.CodePays=M.codePays;
```

| nomAuteur | prenomAuteur | CodePays | libelle       |
|-----------|--------------|----------|---------------|
| NDIAYE    | Samba        | 33       | France        |
| NDIAYE    | Jeanne       | 221      | Senegal       |
| Diop      | samour       | 222      | Gambie        |
| Gaye      | Mbaye        | 229      | Cote d'Ivoire |
| diop      | Maimouna     | 33       | France        |
| NDIAYE    | Deme         | 221      | Senegal       |
| lo        | mami         | 222      | Gambie        |
| diop      | dieynaba     | 221      | Senegal       |

8 rows in set (0.02 sec)

Figure 9 – Jointure naturelle entre la table Pays et Musique

Dans la requête de la figure 9, on a sélectionné : le nom, prenom, code

### 3.5 Opérateurs IN et NOT IN

pays et libellé de tous les auteurs. Ici, on remarque après la clause **FROM**, que la selection se fait dans les deux tables (Musique et Pays). *CodePays* est clé primaire dans la table Pays et clé étrangère dans la table Musique alors la jointure est bien un jointure naturelle. Dans la clause **WHERE**, nous remarquons que la condition se fait dans la colonne *CodePays* des deux tables. Alors, on a un ambiguïté entre ces deux attributs qui portent le même nom. C'est pourquoi, on a renommé les deux tables **Musique** par M et **Pays** par P. Dans la clause WHERE :

+ *P.CodePays* représente la colonne CodePays de la table **Pays**,

+ *M.CodePays* représente la colonne CodePays de la table **Musique**

### 3.5 Opérateurs IN et NOT IN

Elles sont des opérations qui testent l'appartenance de l'élément à un ensemble :

- 
- + Soit en extension  $\Rightarrow$  ensemble de valeurs entre parenthèses
    - + Exemple 1 : Recherche de l'ensemble de auteur dans la table Musique qui ont comme de famille "Gaye" ou "Sall"
    - + `SELECT * FROM Musique WHERE NomAuteur IN ( "Gaye","Sall");`
  - + Soit un **SELECT** entre parenthèses dont le résultat ne doit faire qu'une seule colonne.

### 3.6 Opérateurs IS NULL et IS NOT NULL

Elles sont des opérations qui testent permet de tester qu'un champ a ( ou n'a pas) la valeur NULL. Attention, NULL est une valeur différente de zéro.

- + NULL est utilisable pour tout type de champ +  
signification : valeur du champ indéfinie
  - + Sélectionner tous les films qui n'ont pas de musiques
  - + `SELECT * FROM Film WHERE IdMusique IS NULL;`

### 3.7 Opérateurs ensemblistes

Quatre opérateurs sont disponibles :

- + **Union** : union de 2 attributs (ou ensembles d'attributs) de même structure (mêmes noms de colonnes)
- + **Intersection** : intersection de 2 ensembles d'attributs de même structure

### 3.8 Les fonctions de groupe ou d'agrégation

- + **Except** : différence de 2 ensembles d'attributs de même structure.
- + **Exists** : permet de vérifier si le résultat d'une requête est vide ou non

### 3.8 Les fonctions de groupe ou d'agrégation

Ces fonctions ne peuvent apparaître que dans les clause **SELECT** ou **HAVING** d'une requête. Nous avons les fonctions suivantes.

- + **Count**(expression) : nombre de lignes qui vérifient expression

+ Exemple 1 : Donner le nombre Auteur dans la table Musique.

+ `Select Count(IdMusique) As NbAuteur From Musique;`

Dans cette requête, on compte le nombre *IdMusique* qui sont dans la table Musique. On aurait le même résultat si on avait choisi un autre attribut ou-bien mettre simplement **count(\*)**. L'option AS est facultative, elle permet de renommer la colonne temporaire du résultat. Dans la figure ci-dessous, on le résultat de cette requête et l'état de la table Musique.

```

+-----+-----+-----+-----+-----+
| IdMusique | nomAuteur | prenomAuteur | emailAuteur | CodePays |
+-----+-----+-----+-----+-----+
| 145 | NDIAYE | Samba | Samba@gmail.com | 33 |
| 146 | NDIAYE | Jeanne | jeanne@gmail.com | 221 |
| 149 | Diop | samour | Samour@gmail.com | 222 |
| 130 | Gaye | Mbaye | gaye@gmail.com | 229 |
| 131 | diop | Maimouna | Samba@gmail.com | 33 |
| 132 | NDIAYE | Deme | deme@gmail.com | 221 |
| 133 | lo | mami | Samba@gmail.com | 222 |
| 134 | diop | dieynaba | dieynaba@gmail.com | 221 |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> select count(IdMusique) AS NbAuteur from musique;
+-----+
| NbAuteur |
+-----+
| 8 |
+-----+
1 row in set (0.00 sec)

```

Figure 10 – Le nombre de ligne de la table Musique

+ Exemple 2 : Donner les nom de familles des auteurs et leur nombre d'occurrence

+ `Select NomAuteur, Count(NomAuteur) As NbAuteur From Musique Group By NomAuteur;`

Dans la figure ci-dessous, on le nom des auteurs des auteurs et leur nombre d'apparition.

### 3.8 Les fonctions de groupe ou d'agrégation

```
mysql> select NomAuteur,count(IdMusique) AS NbAuteur from musique
-> Group By NomAuteur;
+-----+-----+
| NomAuteur | NbAuteur |
+-----+-----+
| Diop      | 3        |
| Gaye      | 1        |
| lo        | 1        |
| NDIAYE    | 3        |
+-----+-----+
4 rows in set (0.06 sec)
```

Figure 11 – Nom de famille dans la table Musique et leur nombre d'occurrence

- + **avg**(expression) : renvoie la moyenne de expression (type numérique) calculée sur les lignes de la table où elle est définie (NULL pas pris en compte).
  - + Sa syntaxe est :
  - + **SELECT AVG(attribut) FROM Table;**
- + **Max** [min](expression) : renvoie la plus grande [petite] valeur de expression sur la table. + Sa syntaxe est :
  - + **SELECT Min(attribut) FROM table;**
- + **sum**(expression) : renvoie la somme de expression (type numérique) calculée sur les lignes de la table où elle est définie (NULL pas pris en compte).
  - + Sa syntaxe est :
  - + **SELECT SUM(attribut) FROM table;**